

# Mais *geradores*

Depois do `yield` básico, os geradores ganham superpoderes: `yield from` delega para outro iterável, um `while True` cria sequências infinitas, vários geradores encadeados formam um **pipeline** que processa um item por vez, e `send()`, `return` e `close()` permitem conversar com o gerador. Nesta aula: cada um deles com um exemplo curto.

`yield from`

Geradores infinitos

Pipeline

`send()`

`return` no gerador

`close()` · `finally`

Formação Claude Code 2026 · Baseado em docs.python.org (Reference: Yield expressions, generator-iterator methods; PEP 380 e PEP 342; itertools), consultados em outubro de 2026 · Exemplos testados com Python 3.14 em `exemplos/` · Saídas reais

## Delegar para *outro iterável*

yield\_from.py

```
def letras():  
    yield from "ab"          # delega para outro iterável  
    yield from ["c", "d"]  
  
def tudo():  
    yield 1  
    yield from letras()      # delega para outro gerador  
    yield 2  
  
print(list(tudo()))
```

SAÍDA

```
[1, 'a', 'b', 'c', 'd', 2]
```

- ✓ `yield from x` entrega, um por um, todos os itens de `x`
- ✓ É o mesmo que `for item in x: yield item`, só que mais curto
- ✓ Funciona com qualquer iterável: string, lista e outro gerador

## Geradores *sem fim*

infinito.py

```
from itertools import islice

def ids(prefixo):
    n = 1
    while True:
        # nunca termina sozinho
        yield f"{prefixo}-{n:03d}"
        n += 1

gerar = ids("PED")
print(next(gerar), next(gerar))
print(list(islice(gerar, 3))) # pega só os próximos 3
```

SAÍDA

```
PED-001 PED-002
['PED-003', 'PED-004', 'PED-005']
```

- ✓ Um `while True` com `yield` não trava: só calcula quando alguém pede
- ✓ Útil para IDs, senhas temporárias, sequências numéricas
- ✓ `islice` continua do ponto atual: depois de `PED-002`, vem `PED-003`

Nunca passe um gerador infinito direto para `list()` ou `sum()` : o programa não termina.

## Geradores *em sequência*

pipeline.py

```
linhas = ["10", "", "7", "x", "3", "25"]

def nao_vazias(itens):
    for item in itens:
        if item.strip():
            yield item

def numeros(itens):
    for item in itens:
        if item.isdigit():
            yield int(item)

def acima_de(limite, nums):
    for n in nums:
        if n > limite:
            yield n

etapas = acima_de(5, numeros(nao_vazias(linhas)))
print(list(etapas))
```

**NAO\_VAZIAS**

tira linhas vazias



**NUMEROS**

converte o que é  
número



**ACIMA\_DE**

filtra pelo limite

- ✓ Cada etapa recebe um iterável e devolve outro
- ✓ Os itens passam **um por vez** pelas três etapas: nada de listas intermediárias
- ✓ Cada função é pequena, testável e reaproveitável

SAÍDA

[10, 7, 25]

## Mandando valores *para dentro*

send.py

```
def media_movel():
    total = quantidade = 0
    media = None
    while True:
        valor = yield media      # recebe o que vier no send()
        total += valor
        quantidade += 1
        media = total / quantidade

m = media_movel()
next(m)                # avança até o primeiro yield
print(m.send(10))
print(m.send(20))
print(m.send(60))
```

SAÍDA

```
10.0
15.0
30.0
```

- ✓ `valor = yield media` faz duas coisas: entrega `media` e espera um valor
- ✓ `m.send(10)` retoma o gerador com `valor = 10` e devolve o próximo `yield`
- ✓ O primeiro `next(m)` é obrigatório: leva o gerador até o primeiro `yield`

O gerador guarda `total` e `quantidade` entre as chamadas, sem variável global nem classe.

## O gerador também *devolve um resultado*

retorno.py

```
def ler_notas(notas):
    validas = 0
    for nota in notas:
        if 0 <= nota <= 10:
            validas += 1
            yield nota
    return validas          # vira o resultado do yield from

def relatorio(notas):
    qtd = yield from ler_notas(notas)
    print("notas válidas:", qtd)

print(list(relatorio([8, 11, 6, -1, 9])))
```

SAÍDA

```
notas válidas: 3
[8, 6, 9]
```

- ✓ O `return` de um gerador encerra a iteração
- ✓ O valor devolvido chega como resultado do `yield from`
- ✓ "notas válidas" aparece primeiro porque o `print` roda quando o `list()` esgota o gerador, antes de a lista ser impressa

## Fechando *com limpeza*

fechar.py

```
def conexao():  
    print("abriu")  
    try:  
        while True:  
            yield "dado"  
    finally:  
        print("fechou")      # roda quando o gerador é fechado  
  
g = conexao()  
print(next(g))  
print(next(g))  
g.close()
```

SAÍDA

```
abriu  
dado  
dado  
fechou
```

- ✓ `g.close()` interrompe o gerador na pausa atual
- ✓ O bloco `finally` roda: lugar certo para fechar arquivos e conexões
- ✓ Depois de fechado, um novo `next(g)` lança `StopIteration`

## Fibonacci *infinito*

**Enunciado:** crie um gerador infinito com a sequência de Fibonacci e mostre os 10 primeiros termos e os menores que 100.

Uma resposta possível

```
from itertools import islice, takewhile

def fibonacci():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b

print(list(islice(fibonacci(), 10)))
print(list(takewhile(lambda n: n < 100, fibonacci())))
```

SAÍDA

```
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34]
[0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89]
```

- ✓ `islice` limita pela quantidade; `takewhile`, por uma condição
- ✓ O gerador nunca termina: quem decide parar é quem consome
- ✓ Desafio: um gerador que entrega só os termos pares

## Teste o que *ficou*

1. `list(g())` com `def g(): yield from "hi"` vale:

A) `["hi"]`

B) `["h", "i"]`

C) `"hi"`

D) `[]`

B.

2. Antes do primeiro `send(valor)`, é preciso:

A) Chamar `close()`

B) Chamar `next()` (ou `send(None)`)

C) Usar `yield from`

D) Nada

B.

3. Como limitar um gerador infinito aos 5 primeiros itens?

A) `list(g)[:5]`

B) `islice(g, 5)`

C) `g[:5]`

D) `len(g) = 5`

B.

4. O valor do `return` de um gerador chega em:

A) No último `next()`

B) No resultado do `yield from`

C) Em `list(g)`

D) Em lugar nenhum

B.

# Geradores que *conversam*

`yield from` delega para outro iterável e recebe o `return` do gerador interno. Um `while True` gera sem fim, e quem consome decide parar com `islice` ou `takewhile`.

Geradores encadeados formam pipelines sem listas intermediárias; `send()` manda valores para dentro e `close()` encerra rodando o `finally`.

`yield from`

`while True`

Pipeline

`send()`

`close()`

Próxima aula → 05 · Aplicações Práticas

Fontes: [docs.python.org/3/reference/expressions.html#yield-expressions](https://docs.python.org/3/reference/expressions.html#yield-expressions); [docs.python.org/3/reference/expressions.html#generator-iterator-methods](https://docs.python.org/3/reference/expressions.html#generator-iterator-methods); [peps.python.org/pep-0380](https://peps.python.org/pep-0380/); [peps.python.org/pep-0342](https://peps.python.org/pep-0342/); [docs.python.org/3/library/itertools.html](https://docs.python.org/3/library/itertools.html) (`islice`, `takewhile`), consultados em outubro de 2026. Exemplos em [exemplos/](#), executados com Python 3.14.4.